SEASUG Paper 34-2026

Closing the Loop: A Portable, Metadata-Driven Framework for Automating EDC Skip Logic Cleaning Across Therapeutic Areas

Chaitanya Popuri, Penumbra, Inc.

ABSTRACT

In a cloud-based electronic data capture (EDC) system, a critical data integrity gap may arise when sites retroactively modify trigger fields: previously entered child data can become hidden in the user interface while remaining active in database exports. This behavior creates orphaned records in which parent responses no longer support the related child data and can require labor-intensive review of exported datasets. This paper presents a closed-loop data cleaning framework that detects and remediates these anomalies using EDC metadata exports. The framework has two phases. First, an automated SAS® macro ingests metadata, dynamically maps parent-child dependencies and flags orphaned records. Second, a correction utility consumes the reviewed detection output and programmatically nullifies invalid hidden data according to variable type. Deployed on two projects in different therapeutic areas, the framework scanned 2,005 variables across 62 case report forms and identified 102 complex logic issues that were confirmed through data management review. The implementation was estimated to save more than 60 hours of manual programming. Because the framework relies on consistently structured EDC metadata, adaptation between therapeutic areas requires minimal updates. Readers will learn how Metadata can support a portable, traceable process for addressing skip logic limitations and improving exported-data integrity.

INTRODUCTION

Skip logic improves the usability of an EDC form by showing or hiding questions according to previously entered responses. A parent or trigger field controls whether one or more child fields are applicable. When the parent response changes after child data has already been entered, the child values may disappear from the site-facing form but remain present in the exported database. The user interface therefore appears consistent even though the export contains values that are no longer supported by the current parent response.

This paper describes a SAS-based framework that utilizes EDC metadata to automate the detection and remediation of hidden skip logic issues. The framework converts skiplogic definitions into parent-child relationships, scans affected export datasets, creates a reviewable issue listing, applies approved nullifications to a controlled copy of the raw data and verifies the resulting differences. Rather than addressing study-specific rules, the framework demonstrates how standardized metadata can support a reusable and scalable data cleaning process across forms and projects.

THE DATA INTEGRITY GAP

A skip logic anomaly occurs when the current parent response indicates that a child field is not applicable, but the exported child variable still contains a value. Table 1 illustrates a simplified example. The current parent response is No, so the imaging date is hidden and should not remain active in the export. The retained date is an orphaned child value.

Role	Example field	Exported value	Interpretation
Parent	Was imaging performed?	No	Current response suppresses the child field.
Child	Imaging date	01MAY2026	Previously entered value remains in the export.

Table 1. Simplified Orphaned Child-Data Example

Manual identification is difficult because reviewers must interpret the parent response, locate every controlled child variable, account for repeat instances and then inspect the exported values. The burden increases when one parent controls several child variables. This framework treats the metadata as the authoritative map for those relationships and uses the map to drive a consistent scan.

CLOSED-LOOP FRAMEWORK

The framework contains five connected stages. The output of each stage becomes the controlled input to the next stage, which provides a traceable path from metadata through correction (Figure 1).

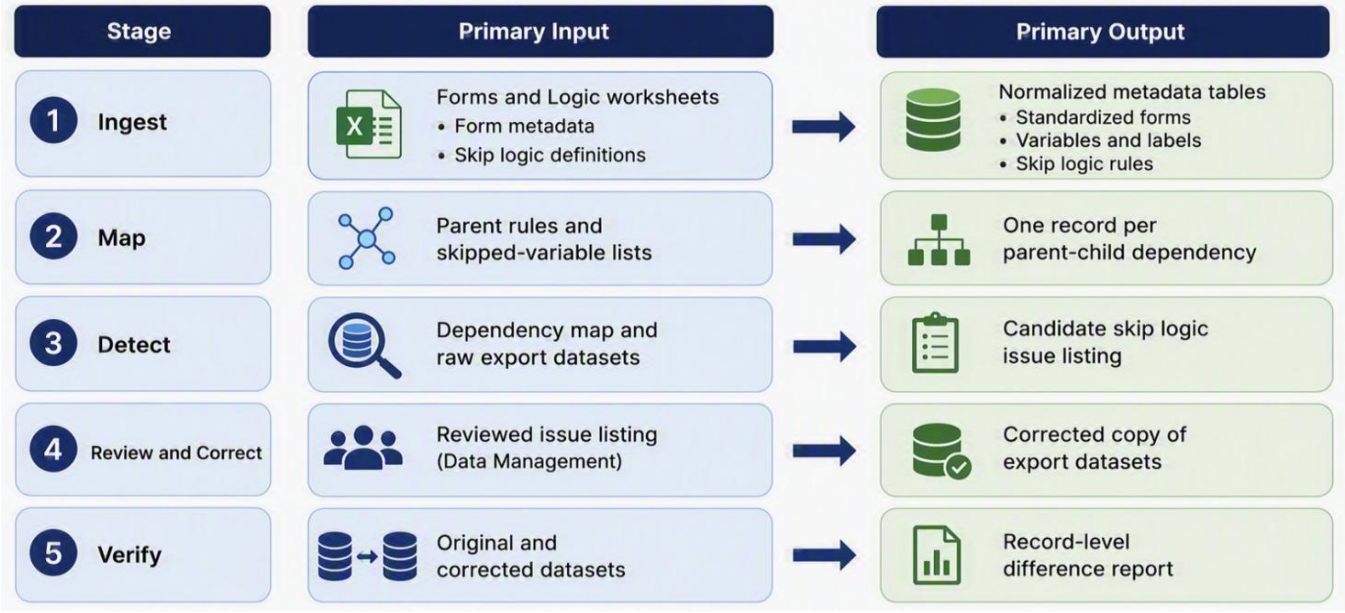


Figure 1. Closed-Loop Processing Stages

METADATA INPUTS

The framework imports two worksheets from the EDC metadata export. The Exports worksheet provides variable-level metadata, while the Logic worksheet provides the skip-logic definitions used to establish parent-child dependencies (Table 2).

Worksheet	Captured Metadata
Exports	Form names, export datasets, variable names, labels, display types, data types
Logic	Parent variables, evaluation conditions, skipped child variables

Table 2. Importing the EDC Metadata Worksheets

DEPENDENCY MAPPING

The program merges exports metadata with logic metadata by form and parent variable. It removes incomplete rows, counts comma-delimited child variables, expands the child list into an array and transposes the array. The resulting control table contains one observation for each parent-child relationship. This normalized structure allows one parent field to control any number of child fields without requiring a separate program for every relationship (Output 1).

```
skipped=compress(skip_field_manual);
countskip=countw(skipped,',');

array skipl(&sk.) $20;
do i=1 to dim(skipl);
   skipl[i]=scan(skipped,i,',');
end;
```

Output 1. Expanding the Skipped-Variable List into Parent-Child Dependencies

DYNAMIC DATASET PROCESSING

The sql procedure creates a distinct list of affected export datasets from the normalized metadata. A macro loop then processes each listed dataset. This design prevents the workflow from relying on a fixed inventory of case report forms. Because the processing list is derived directly from metadata, new forms and skip logic dependencies can be incorporated without modifying the core framework (Output 2).

```
proc sql;
  select distinct upcase(export)
    into :dsnlist separated by ","
  from combineforms;
quit;

%do %while (%scan(%nrbquote(&dsnlist),&i,",") ne %str());
  %let dsn=%scan(%nrbquote(&dsnlist),&i,",");
  /*process dataset*/
%end;
```

Output 2. Metadata-Driven Dynamic Dataset Processing

DETECTION METHODS

The detection utility joins display information back to the dependency map and separates rules into three processing paths. This separation is important because a single comparison does not represent every EDC field type or condition pattern (Table 3).

Processing path	Metadata indicator	Anomaly tested
Standard categorical rule	Non-checkbox child and explicit category condition	Parent equals the skip condition while the child contains data.
Continuous rule	Parent field is not missing	Parent variable has non missing value, yet dependent child variables retain historical input despite parent suppression.
Checkbox rule	Checkbox display type	Parent equals the skip condition while the child contains a response other than No or blank.

Table 3. Detection Paths Derived from Metadata

DYNAMIC RULE EXECUTION

Within each path, the control table is loaded into macro variables that represent the parent variable, category condition and child variable. The generated DATA step evaluates every applicable relationship against the current export dataset. When a rule identifies an orphaned value, the program creates a response string containing the parent label, parent variable, parent response, child label, child variable and child response. SAS functions such as VLABEL, VNAME and VVALUE allow the listing to retain both machine-readable variable names and reviewer-friendly context (Output 3).

```
if compress(&parent.)=condition and &child. ne '' then
       response=vlabel(&parent.)||' $'||vname(&parent.)||' $'||
                    strip(vvalue(&parent.))||'#'||vlabel(&child.)||' $'||
                        vname(&child.)||' $'||strip(vvalue(&child.));
```

Output 3. Representative Parent-Child Detection Pattern

REPEAT-FORM TRACEABILITY

Repeating forms require a stable instance identifier so that the detection and correction phases target the same record. The implementation uses SUBJECTID, VISIT, FORMNUMBER and FORM as primary traceability variables. For selected repeating forms, the program derives FORMNUMBER from row-level grouping variables before sorting and transposing the findings. These form-specific normalizations are isolated from the metadata-driven core so that they can be maintained without redesigning the full framework.

REVIEW LISTING

The detection phase consolidates findings across all processed datasets and parses the response string into parent and child components. The final listing presents the form, subject, visit, Form Number, parent details, and child details. Data management can review each candidate against source context and determine whether the hidden child value should be removed. This review step is the control point between automated detection and automated correction (Table 4).

Form	Subject	Visit	Form number	Parent finding	Child finding
Example Form	SUBJ001	Procedure	1	Treatment is not Stenting	Participant has Stent length non missing
Example Form	SUBJ014	Enrollment	1	Patient Enrolled is No	Protocol version is C

Table 4. Detection Listing

CONTROLLED CORRECTION AND VALIDATION

TYPE-AWARE NULLIFICATION

The correction utility reads the reviewed detection output and maps the form name back to its export dataset. It joins each child variable to SASHELP.VCOLUMN to obtain the stored SAS type. For each affected dataset, the utility copies the original observations to a separate output library and applies correction only when the subject, visit, form number, and child variable match an approved finding. Character values are assigned a blank and numeric values are assigned the SAS missing value. Unaffected export datasets are copied without modification.

To execute record level nullification dynamically, the macro binds key identifiers including SubjectID (subj), Visit (vst), Formnumber (frl), data type (dtp) and child variable name (cvar) into sequential macro arrays using CALL SYMPUT, then iterates across all flagged records in the target dataset (Output 4).

```
%let N=0;
data tmp1;
  set COMBFORMS;
  retain nn 0;
  nn+1;
  call symput(compress('vst'||put(nn,3.0)),compress(VISIT));
  call symput(compress('subj'||put(nn,3.0)),compress(SUBJECTID));
  call symput(compress('frl'||put(nn,3.0)),compress(FORMNUMBER));
  call symput(compress('dtp'||put(nn,3.0)),compress(dtype));
  call symput(compress('cvar'||put(nn,3.0)),compress(chvar));
  call symput('N',compress(put(nn,3.0)));
run;

data RAWN.&dsn.(drop=vst subj frl dtp);
  set old_&dsn.;
  %local ii;
  %do ii=1 %to &N.;
    length vst $40 subj $20 frl dtp $10 ;
    vst="&&vst&ii.";
    subj="&&subj&ii.";
    frl="&&frl&ii.";
    dtp="&&dtp&ii.";
        if compress(visit)=vst and subjectid=subj and formnumber=frl
        and dtp="char" then &&cvar&ii.=" ";
        else if compress(visit)=vst and subjectid=subj and formnumber=frl
        and dtp="num" then &&cvar&ii.=.;
  %end;
run;
```

Output 4. Type-Aware Nullification Pattern

DATASET COMPARISON

The final phase compares each original export dataset with its corrected counterpart by SUBJECTID, VISIT, and FORMNUMBER. Before comparison, the same repeat-label normalization is applied to both versions. The COMPARE procedure writes unequal observations to dataset-specific outputs, and the program consolidates the affected dataset and record identifiers into a difference report. This step confirms that the controlled copy differs from the original where nullification was intended and provides a record-level audit aid for downstream review.

IMPLEMENTATION RESULTS

The framework was deployed on two projects in different therapeutic areas. Across those implementations, it scanned 2,005 variables from 62 case report forms. The detection and review process identified 102 complex logic issues, and data management confirmed the resulting corrections. The estimated reduction in manual programming effort exceeded 60 hours. Table 5 summarizes the reported implementation measures .

Measure	Observed result
Projects	2
Therapeutic areas	2
Case report forms scanned	62
Variables scanned	2,005
Complex logic issues identified and resolved	102
Estimated manual programming effort avoided	60+ hours

Table 5. Implementation Results

Porting the framework between therapeutic areas required minimal updates because the EDC metadata followed a consistent structure. The reusable components included metadata import, dependency expansion, dynamic dataset selection, detection paths, review-listing construction, type-aware correction and dataset comparison. Local updates were primarily associated with data dictionary names, directory configuration, excluded technical datasets and repeat-form normalization.

CONCLUSION

Hidden child values created by retroactive trigger-field changes can leave exported clinical data inconsistent with the current EDC form state. The closed-loop framework presented in this paper converts EDC metadata into a reusable control layer for detection, review, correction, and verification. By dynamically mapping dependencies, separating common rule patterns, preserving record traceability, applying type-aware nullification, comparing original and corrected datasets the framework addresses both anomaly identification and remediation. Its deployment across two therapeutic areas demonstrates that consistently structured metadata can support portable automation, measurable effort reduction and stronger exported-data integrity.

ACKNOWLEDGMENTS

The author gratefully acknowledges the data management and statistical programming colleagues at Penumbra, Inc., who reviewed the detected findings, validated the output listings and supported the deployment of the framework across both clinical studies. All clinical trial examples, variable identifiers and system directory paths in this paper have been generalized or anonymized for conference publication.

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Chaitanya Popuri
Penumbra, Inc.

One Penumbra Place

Alameda, CA 94502

E-mail: cpopuri@penumbrainc.com

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Other brand and product names are trademarks of their respective companies.